Do We Really Need Quantum Machine Learning?: A Multidimensional Empirical Study

Sudip Vhaduri[e1], Ryan Gammon[e2], and Sayanton Dibbo[*1]
[1]Department of Computer Science, University of Alabama, AL 35487
[1]{svhaduri,sdibbo}@ua.edu, [2]rdgammon@crimson.ua.edu

***Abstract*—The rapid growth of computer vision and increasingly complex image recognition tasks has exposed fundamental computational limitations of classical machine learning models, motivating the exploration of quantum computing as an emerging new paradigm. This paper presents a comprehensive benchmarking study of classical and quantum machine learning models for image recognition on the MNIST handwritten digit dataset, evaluating both traditional models, a Classical Support Vector Machine (CSVM) and a Quantum Support Vector Machine (QSVM), and deep neural network models, a Classical Convolutional Neural Network (CCNN) and a Quantum Convolutional Neural Network (QCNN), across four performance dimensions: classification accuracy, computational runtime, parameter count, and memory requirements. Experiments are conducted as functions of both feature dimensionality and sample size, and across CPU and GPU execution environments, providing a controlled, multidimensional comparison to address gaps in prior work. For the SVM-based models, QSVM consistently outperforms CSVM in accuracy, reaching $\sim$ 0.90 versus $\sim$ 0.85 at 1,000 samples, with a higher computational cost. A feature count of 10 qubits and a sample size in the range of 200 – 500 emerge as practical operating points that balance accuracy and runtime. For the neural network models, CCNN and QCNN achieve comparable classification accuracy, both exceeding 0.96 at 64 features and 60,000 samples, yet QCNN offers substantially superior parameter and memory efficiency, requiring $\sim$ 94% fewer parameters and $\sim$ 75% less memory than CCNN at higher feature counts, while incurring higher runtime. Across both model families, quantum models consistently outperform classical models by greater margins in accuracy as feature dimensionality or sample size increases, suggesting that quantum models offer the greatest relative advantage in high-data or high-dimensionality regimes. These findings provide actionable guidance for practitioners considering quantum machine learning for computer vision tasks under real-world resource constraints, and directly inform the design of efficient, quantum-ready perception systems for the automated, connected, and safe transportation applications central to the mission of the Alabama Transportation Institute (ATI) and the National Center for Transportation Cybersecurity and Resiliency (TraCR).**

***Index Terms*—Benchmarking, Classification Accuracy, Convolutional Neural Network, GPU Acceleration, Handwritten Digit Recognition, Image Classification, MNIST, Quantum Computing, Quantum Convolutional Neural Network, Quantum Kernel, Quantum Machine Learning, Support Vector Machine.**

I. Introduction

Computer vision and image recognition have become indispensable pillars of modern artificial intelligence, underpinning transformative applications in autonomous vehicles, medical diagnostics, industrial quality control, and surveillance [1]–[6]. The global computer vision market was valued at approximately $19.82 billion in 2024 and is projected to reach $58.29 billion by 2030, growing at a compound annual growth rate (CAGR) of 19.8% [7]. Classical machine learning paradigms, most prominently Support Vector Machines (SVMs) and Convolutional Neural Networks (CNNs), have long driven progress in image classification tasks [8], [9]. However, as visual datasets scale into the hundreds of millions and recognition tasks grow increasingly complex, these classical approaches confront fundamental computational bottlenecks: SVMs suffer from complex kernel construction, while deep CNNs demand hundreds of millions of parameters and substantial memory footprints, rendering large-scale deployment computationally prohibitive [8]. Such constraints have motivated the search for radically different computing substrates capable of representing data more efficiently and capturing complex feature interactions that challenge classical models.

Quantum computing has emerged as a compelling new paradigm, with the global quantum computing market estimated at $1.42 billion in 2024 and projected to reach $4.24 billion by 2030 at a CAGR of 20.5% [10]. Quantum machine learning (QML) exploits the principles of superposition and entanglement to represent data in exponentially high-dimensional Hilbert spaces, offering the potential for fundamentally more expressive feature maps than their classical counterparts [9], [11], [12].

This work is motivated in part by the applied transportation research mission of the Alabama Transportation Institute (ATI) at The University of Alabama, which is committed to achieving *ACES*^{2} *Mobility*— Automated, Connected, Electric, Shared, and Safe transportation—for the residents of Alabama and beyond [13]. Computer vision is a foundational enabling technology for this vision: autonomous perception, real-time traffic monitoring, and vehicle classification in connected and autonomous vehicles (CAVs) all rely critically on fast, accurate, and resource-efficient image recognition models. Additionally, this work aligns with the mission of the National Center for Transportation Cybersecurity and Resiliency (TraCR), headquartered at Clemson University, which identifies quantum computing and artificial intelligence as key technologies for hardening transportation cyber-physical systems against emerging threats [14]. By systematically benchmarking classical and quantum machine learning models across accuracy, runtime, parameter count, and memory, this paper aims to

* Corresponding author. e equal contribution.

provide the transportation research community with concrete empirical guidance on whether, and under what conditions, quantum models offer a practical advantage for the vision and cybersecurity workloads of next-generation intelligent transportation systems.

II. Related Work

Due to the ever-growing importance of computer vision and image recognition/classification, researchers have been using various classical machine learning and deep learning modeling techniques, such as support vector machines, convolutional neural networks, and others [1]–[6]. As recognition tasks become more complex and require multidimensional improvements, the classical techniques struggle due to their inherent shortcomings. Therefore, researchers have started exploring the recent emerging computing paradigm, i.e., quantum machine learning, in computer vision and image recognition, like other areas [15], due to its ability to represent data in high-dimensional Hilbert spaces [11] and capture complex interactions through quantum entanglement [12].

In this new era of quantum computing, while some efforts have been made to develop a quantum computing-based generative model of synthetic image generation [16], image recognition has received limited attention, with some scattered and independent efforts being made either to achieve a better accuracy with fewer parameters in deep neural networks, such as the convolutional neural networks [17], [18], to reduce quantum bit storage in traditional machine learning (ML) models, such as the support vector machines [19], or simply to achieve a higher accuracy [20]. Yet other critical evaluation metrics, such as computational runtime and scalability, often received little attention.

However, while these quantum machine learning works primarily focus on the accuracy or at most one secondary metric, like the number of parameters or quantum bit storage for either traditional ML models or deep neural network models, current research lacks a comprehensive comparison between quantum and classical image recognition using both traditional machine learning and deep neural network models across multidimensional evaluations simultaneously. In particular, we found no studies that jointly analyze accuracy, runtime, parameter efficiency, and memory/storage requirements as functions of either feature dimensionality or sample size, under comparable experimental conditions such as CPU or GPU. **Our work aims** to address these gaps by systematically evaluating these metrics as joint functions of sample size and feature dimensionality across both traditional machine learning models, such as SVMs, and deep neural network models, such as CNNs, and by additionally comparing performance across CPU and GPU execution environments.

III. Methods

In this section, we present the MNIST dataset used in this paper (Section III-B), the set of machine learning models (Section III-C), and our experimental setup (Section III-D). Before presenting the details, we will outline the performance measures used in this work.

A. Preliminaries: Performance Metrics

In this work, we consider the following metrics to evaluate the performance of different modeling schemes/setups:

Accuracy is the fraction of correct classification/identification, defined as below,

$$Accuracy = \frac{\sum_{i=1}^{N} c_i}{N} \tag{1}$$

Where,

$$c_i = \begin{cases} 1 & \text{if } i^{th} \text{ classification is correct} \\ 0 & \text{otherwise} \end{cases}$$

And, N is the total number of classifications in a multi-class classification task. We also use ***runtime***, in seconds (s), to compare the performance of models. Further, for the deep neural networks, we consider the total number of ***parameters*** and the ***memory*** requirements in kilobytes (kB).

B. Image Dataset

The experiments were conducted using the MNIST [21] handwritten digit dataset, accessed through the `fetch_openml` function from scikit-learn. MNIST contains 70,000 grayscale handwritten digits labeled 0 – 9. In our experiment, we use scikit-learn's `train_test_split` to use 75% for training and 25% for testing. The original dimensionality of these grayscale images is 28 x 28 = 784. The pixel values are normalized, resulting in each having a value of 0 to 1, representing how shaded the region is.

C. Machine Learning Models

In this paper, we experiment with traditional and quantum machine learning models described below.

1) Classical Support Vector Machine (SVM) Baseline: For the classical SVM (CSVM) baseline, we use the Support Vector Machine implementation provided by `scikit-learn` (`SVC`). The MNIST dataset is first truncated to a fixed number of samples and reduced in dimensionality using Principal Component Analysis (PCA), where the number of principal components is treated as a tunable hyperparameter. After dimensionality reduction, the features are standardized using `StandardScaler`. The dataset is then split into training and testing sets with a fixed random seed for reproducibility.

We evaluate a Support Vector Machine using a precomputed cosine similarity kernel. Specifically, the kernel matrix is explicitly constructed using pairwise cosine similarities between feature vectors and passed to the SVM as a precomputed Gram matrix. Cosine similarity between two feature vectors $\mathbf{A}$ and $\mathbf{B}$ is defined as:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$

Using this measure, we construct the Gram matrix $G \in \mathbb{R}^{n \times n}$, where each entry corresponds to the cosine similarity between two samples:

$$G_{ij} = \cos(x_i, x_j)$$

$$G = \begin{bmatrix} \cos(x_1, x_1) & \cos(x_1, x_2) & \cdots & \cos(x_1, x_n) \\ \cos(x_2, x_1) & \cos(x_2, x_2) & \cdots & \cos(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ \cos(x_n, x_1) & \cos(x_n, x_2) & \cdots & \cos(x_n, x_n) \end{bmatrix}$$

2) Quantum Support Vector Machine (QSVM): The Quantum Support Vector Machine (QSVM) uses the same preprocessing pipeline and experimental configurations as the classical SVM (CSVM), including PCA-based dimensionality reduction and standardization. The number of PCA components is set equal to the number of qubits in the quantum model, ensuring that each classical feature vector matches the dimensionality required for quantum encoding.

Formally, after dimensionality reduction, each input sample is represented as:

$$x_i \in \mathbb{R}^d, \quad d = n_{\text{qubits}}$$

Classical feature vectors are encoded into quantum states using an angle embedding function implemented in `PennyLane`. This embedding maps classical data to a parameterized quantum state:

$$x_i \mapsto |\psi(x_i)\rangle$$

The quantum feature map is realized through rotation-based encoding, where each feature is encoded into qubit rotation angles.

The quantum kernel is then defined as the overlap between quantum states:

$$K(x_i, x_j) = |\langle \psi(x_i) | \psi(x_j) \rangle|^2$$

This kernel is used to construct the Gram matrix:

$$K_{ij} = |\langle \psi(x_i) | \psi(x_j) \rangle|^2$$

which is passed to the SVM for classification in the same manner as the classical kernel.

We use the cosine similarity kernel as a classical baseline for comparison with the quantum kernel. Both approaches are based on inner products between feature representations; however, cosine similarity operates in a classical Euclidean feature space, whereas the quantum kernel is defined as the inner product between quantum states in a high-dimensional Hilbert space.

3) Classical Convolutional Neural Network (CNN) Baseline: We implement a fully classical CNN (CCNN) in PyTorch to provide a baseline for comparison with the quantum neural network. The model follows a bottleneck architecture designed to match the dimensionality of the quantum model for a fair comparison.

Let each input image be denoted as:

$$x_i \in \mathbb{R}^{28 \times 28}$$

The first stage performs feature compression by flattening the image and applying a learned linear projection:

$$z_i = W_r x_i + b_r, \quad z_i \in \mathbb{R}^d$$

where $d \in \{16, 32, 64\}$ is the reduced feature dimension used for comparison with the quantum CNN.

This dimensionality reduction layer plays the same role as PCA in the classical SVM baseline, but is learned directly through backpropagation.

The compressed features are then passed through a fully connected neural network:

$$h_i^{(1)} = \sigma(W_1 z_i + b_1)$$
$$h_i^{(2)} = \sigma(W_2 h_i^{(1)} + b_2)$$

where $\sigma(\cdot)$ denotes the ReLU activation function.

Finally, the output layer produces class logits:

$$\hat{y}_i = W_o h_i^{(2)} + b_o$$

The model is trained using the cross-entropy loss:

$$\mathcal{L} = -\sum_i y_i \log(\hat{y}_i)$$

Optimization is performed using the Adam optimizer with learning rate $\eta = 0.001$. Five epochs are used due to the large computational cost associated with training a quantum neural network.

4) Quantum Convolutional Neural Network (QCNN): The quantum convolutional neural network (QCNN) mirrors the architecture of the classical model, with the key distinction that the classical hidden layers are replaced by variational quantum circuits. This enables a quantum feature transformation of classical data prior to classification.

Let each input image be represented as:

$$x_i \in \mathbb{R}^{28 \times 28}$$

A classical linear projection is first applied to reduce the dimensionality of the input:

$$z_i = W_r x_i + b_r, \quad z_i \in \mathbb{R}^{n_{\text{features}}}$$

The reduced feature vector is then partitioned into c quantum circuits, where:

$$c = \frac{n_{\text{features}}}{q}$$

and $q = 4$ denotes the number of qubits per circuit.

Each feature vector is reshaped as:

$$z_i \to \{z_i^{(1)}, z_i^{(2)}, \ldots, z_i^{(c)}\}, \quad z_i^{(k)} \in \mathbb{R}^q$$

Each sub-vector is encoded into a quantum state using angle embedding:

$$z_i^{(k)} \mapsto |\psi(z_i^{(k)})\rangle$$

The encoding is implemented using single-qubit rotations:

$$R_X(\theta), \quad R_Y(\theta)$$

where the input features are used as rotation angles.

Variational Quantum Circuit: Each quantum circuit consists of a parameterized variational layer with trainable weights:

$$U(\theta) = \prod_{l=1}^{D} U_l(\theta_l)$$

where D is the circuit depth. Each layer applies: - single-qubit rotations - entangling CNOT gates

After applying the variational circuit, the output is measured using Pauli-Z expectation values:

$$f(z_i^{(k)}) = [\langle\psi|Z_1|\psi\rangle, \langle\psi|Z_2|\psi\rangle, \ldots, \langle\psi|Z_q|\psi\rangle]$$

Thus, each quantum circuit produces q real-valued features. The full quantum transformation can be written as:

$$x_i \xrightarrow{\text{reduction}} z_i \xrightarrow{\text{quantum layer}} h_i \in \mathbb{R}^{n_{\text{features}}}$$

where:

$$h_i = \bigcup_{k=1}^{c} f(z_i^{(k)})$$

The final class prediction is computed using a classical linear layer:

$$\hat{y}_i = W_o h_i + b_o$$

The model is trained using the cross-entropy loss:

$$\mathcal{L} = -\sum_i y_i \log(\hat{y}_i)$$

Optimization is performed using the Adam optimizer.

The number of qubits per circuit is fixed to $q = 4$, and the number of circuits is determined by:

$$c = \frac{n_{\text{features}}}{4}$$

We evaluate the model under the same variables as the classical baseline.

D. Experimental Setup

As an overview, the setup includes: i) Python 3.11.7, ii) PyTorch and `torchvision` (with an appropriate CUDA build if using GPU) [22], iii) For quantum simulation, `pennylane` [23] is used, iv) Other Python packages used in the experiment: `pandas` [24], `numpy` [25], `matplotlib` [26], `scikit-learn` [27]. In this work, NVIDIA H100 80GB HBM3 and Tesla V100-PCIE-16GB GPUs were used.

Two experimental settings are considered for the CSVM baseline and QSVM are:

1) Feature dimension scaling: the number of PCA components is varied in {2, 4, 6, 8, 10, 12}, while the number of samples is fixed at 300.
2) Sample size scaling: the number of training samples is varied in {50, 100, 200, 500, 1000}, while the feature dimension is fixed at 12.

The PCA dimension aligns with the quantum model's dimensionality, enabling a controlled comparison between the classical SVM (CSVM) baseline and the quantum model.

Two experimental variables are considered for the CCNN baseline and QCNN are:

1) Feature dimension: In our experiments, we used feature dimensions, $d \in \{16, 32, 64\}$, at a sample size of 60,000.
2) Training sample size: We used $N \in \{15{,}000, 30{,}000, 60{,}000\}$ to evaluate scaling behavior at feature dimension 16.

IV. Results

In this section, we analyze the performance of traditional (Section IV-A) and deep neural network (Section IV-B) models when identifying/classifying the 10 digits in the MNIST dataset using CPU and GPU. To compare the performance of classical and quantum models, we will primarily rely on classification/identification accuracy and runtime.

A. Traditional Models

We will first analyze and compare the performance of Support Vector Machines (SVMs), both the classical SVM (CSVM) and the quantum SVM (QSVM), on CPU and GPU.

1) Effect of Qubit/Feature Count: Figure 1 summarizes our findings when varying qubits/features (i.e., 2, 4, 6, 8, 10, 12), keeping the sample size fixed (i.e., 300).

Accuracy Comparison: In Figure 1a, the accuracy plot (left subplot), we observe that, in general, accuracy improves with the increase of qubits/features for both CSVM and QSVM. For a fixed qubit/feature, the quantum SVM (QSVM) usually outperforms the CSVM.

Runtime Comparison: In Figure 1b, the runtime plot (right subplot), we observe that the runtime is very low and steady for classical SVMs (CSVMs) compared to quantum SVM (QSVM), which shows an increase with the increase of qubits/features. While the QSVM GPU increases linearly, the QSVM CPU increases exponentially.

Accuracy vs. Runtime Trade-off: From Figure 1, we found QSVM on GPUs to be a better choice, among the four SVM and CPU/GPU combinations, due to its higher accuracy and linearly reasonable runtime. Therefore, for the accuracy vs. runtime trade-off analysis, we chose the QSVM on GPUs.

In Figure 2, we observe two crossovers between the dual axis plots of accuracy and runtime at qubit/feature count 2 and 11. However, at qubit/feature count 10, we observe that the accuracy line flattens, while the runtime line continues to sharply rise from qubit/feature count 2 onward. Therefore, qubit/feature count 10 could be an optimal choice to make a trade-off between accuracy and runtime.

Miscellaneous – Accuracy Gap Between QSVM & CSVM: For this analysis, we compare the QSVM and CSVM based on our findings from an accuracy comparison across four combinations of SVM and CPU/GPU (Figure 1a). In the figure, we observe that as the qubit/feature count increases, the accuracy gap between the best QSVM and the best CSVM decreases, with a sharp drop from qubit/feature count 2 to 6, then flattens to 0.06. Therefore, in this setup, after qubit/feature count 6, we observe similar performance between the CSVM and QSVM, with QSVM slightly more accurate.

2) Effect of Sample Size: Figure 3 presents our findings when varying sample sizes (i.e., 50, 100, 200, 500, 1000), keeping the qubits/features fixed (i.e., 12).

Accuracy Comparison: In the accuracy plot in Figure 3a (left subplot), we observe that accuracy improves with the increase of sample size for both CSVM and QSVM. At a sample size of 1000, QSVM achieves the highest ≈ 0.9 accuracy, and CSVM achieves its highest accuracy of ≈ 0.85.

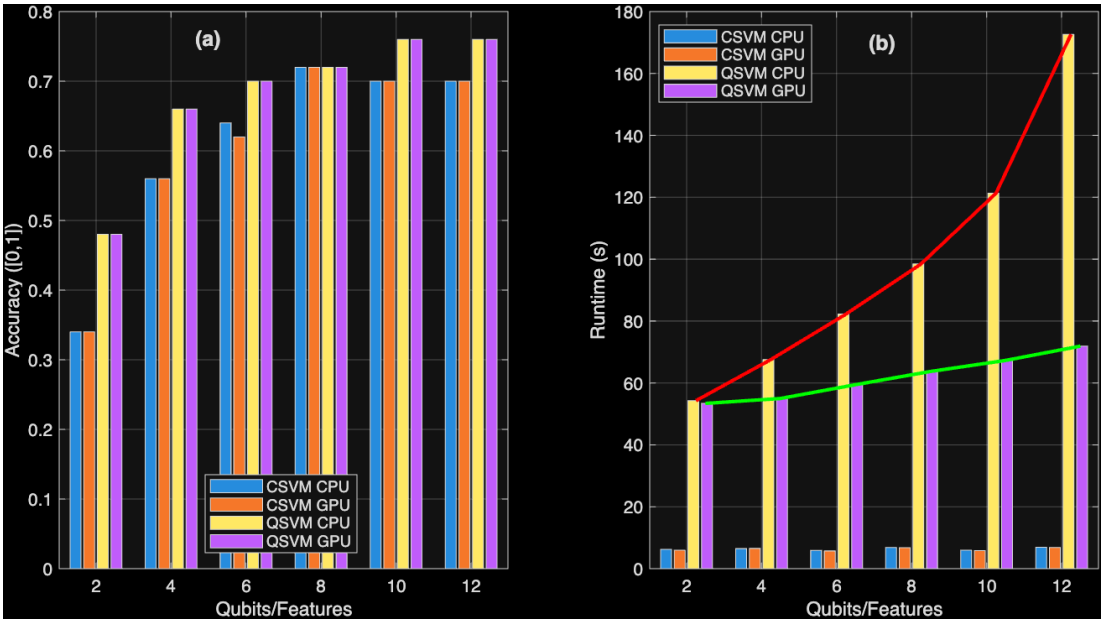


Fig. 1: Bar graph plot of (a) accuracy (left) and (b) runtime (right) variation across the CSVM and QSVM with CPU and GPU when varying qubits/features but keeping sample size fixed (i.e., 300).

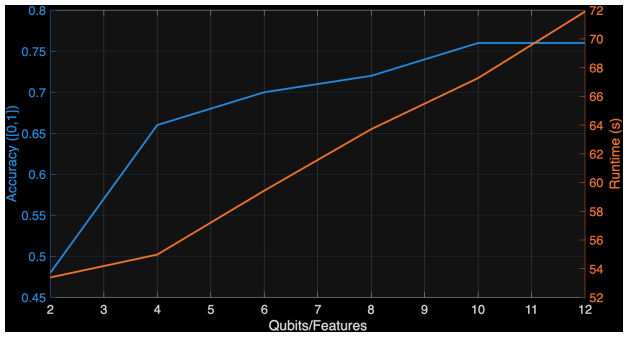


Fig. 2: Accuracy vs. runtime tradeoff of QSVM using GPU varying qubits/features but keeping sample size fixed (i.e., 300).

Runtime Comparison: In the runtime plot in Figure 3b (right subplot), we observe trends similar to those in Section IV-A1 as we varied the qubit/feature count, keeping the sample size fixed. The runtime is very low and steady for classical SVMs (CSVMs), varying from 3.23s to 4.28s (CSVM CPU) and from 6.37s to 8.28s (CSVM GPU); whereas the runtime of quantum SVM (QSVM) increases rapidly with increasing sample size, with QSVM CPU runtime increasing faster than QSVM GPU.

Accuracy vs. Runtime Trade-off: Similar to the findings in Section IV-A1, we found QSVM on GPUs to be a better choice among the four SVMs and CPU/GPU combinations for trade-off analysis, due to its higher accuracy and relatively lower runtime than QSVM on CPU, as found in Figure 3. Therefore, for the accuracy vs. runtime trade-off analysis with varying sample sizes, we chose QSVM on GPUs.

In Figure 4a, we observe two close proximities between the dual-axis plots of accuracy and runtime at around sample sizes 50 and 1000. While accuracy increases sharply up to a sample size of 200, runtime increases sharply beyond 500. Therefore, a range of [200,500] can be an optimal choice for the sample size (at the fixed qubit/feature count of 12) to balance accuracy and runtime.

Miscellaneous – Accuracy Gap Between CPU and GPU versions of CSVM and QSVM: For this analysis, we compare the QSVM and CSVM based on our findings from an accuracy comparison across four combinations of SVM and CPU/GPU (Figure 3a). In that figure, we observed that CSVM accuracy varied across CPUs and GPUs, whereas QSVM achieved similar accuracy across both. Therefore, in addition to comparing the best CSVM and QSVM, we also compare the CSVM on CPU and GPU.

In Figure 4b, we present the accuracy gap between the best CPU and GPU versions of the CSVM as well as the gap between the best CSVM and QSVM. First, in Figure 3, we observed that at a lower sample size (i.e., 50), CSVM on the GPU achieves the highest accuracy among the four models, and as the sample size increases, the other models approach the accuracy close to that of CSVM on the GPU.

In the dashed line in Figure 4b, we observe that as the sample size increases from 50 to 200, the accuracy gap between the CPU and GPU versions of the CSVM decreases faster, and at a sample size of 200, the gap approaches zero and remains steady afterward. Therefore, in this setup with a fixed qubit/feature count of 12, after 200 samples, we would see the same performance across the CPU and GPU versions of the CSVM.

Similarly, though QSVM underperforms CSVM at small sample sizes (negative accuracy gap shown on the solid line

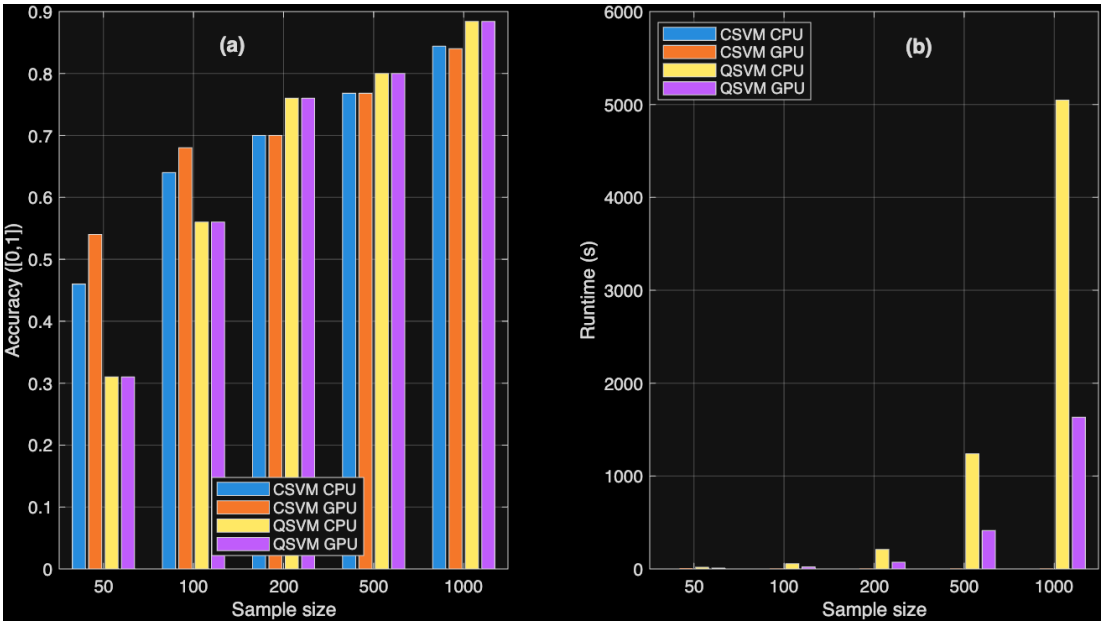
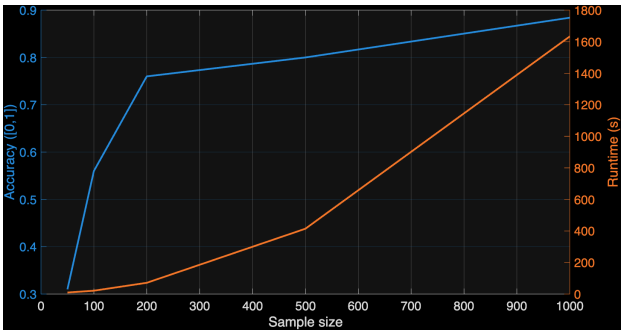
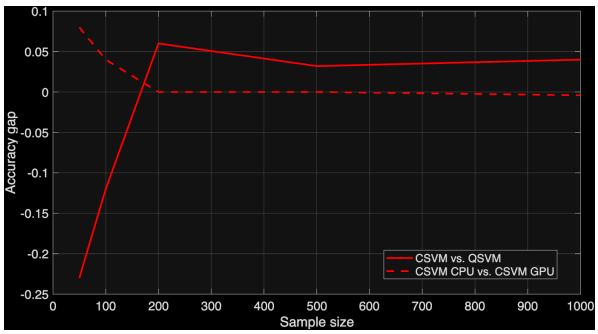


Fig. 3: Bar graph plot of (a) accuracy (left) and (b) runtime (right) variation across the CSVM and QSVM with CPU and GPU when varying sample size but keeping qubits/features fixed (i.e., 12).



(a) Accuracy vs. runtime tradeoff of QSVM in GPU



(b) Accuracy gap among the best QSVM and CSVM (CPU vs. GPU)

Fig. 4: Effect of varying sample size but keeping qubits/features fixed (i.e., 12)

in Figure 4b), as the sample size increases, the gap between QSVM and CSVM shrinks, and after 200 samples, QSVM slightly outperforms CSVM (positive gap value shown on the solid line). Therefore, in this setup with a fixed qubit/feature count of 12, after 200 samples, we would see close performance across the CSVM and QSVM, with QSVM being more accurate.

B. Deep Neural Network Models

In this section, we will analyze and compare the performance of convolutional neural networks (CNNs), both the classical CNN (CCNN) and the quantum CNN (QCNN), on CPU and GPU. For each combination, we vary the epoch count from 1 to 5.

1) Effect of Feature Count: Table I summarizes the performance of different models across different feature counts (i.e., 16, 32, and 64), varying the epochs while keeping

TABLE I: Accuracy variation of QCNN and CCNN across different features and epochs for a fixed sample size of 60,000

Models	Feature Count	Accuracy across 5 epochs				
		1	2	3	4	5
CCNN CPU	16	0.9026	0.918	0.9355	0.9391	0.9434
	32	0.9348	0.9475	0.9529	0.9526	0.964
	64	0.9432	0.9507	0.9633	0.9675	0.9613
CCNN GPU	16	0.9094	0.9237	0.936	0.9386	0.9411
	32	0.9348	0.9486	0.9471	0.9579	0.9584
	64	0.9322	0.9521	0.9606	0.9659	0.968
QCNN CPU	16	0.8921	0.9186	0.9105	0.9209	0.9245
	32	0.9394	0.9378	0.9454	0.9411	0.9458
	64	0.9562	0.9597	0.9627	0.9629	0.9609
QCNN GPU	16	0.8906	0.9141	0.9191	0.9204	0.9273
	32	0.9325	0.9462	0.9487	0.9479	0.949
	64	0.9572	0.9603	0.9616	0.9607	0.9609

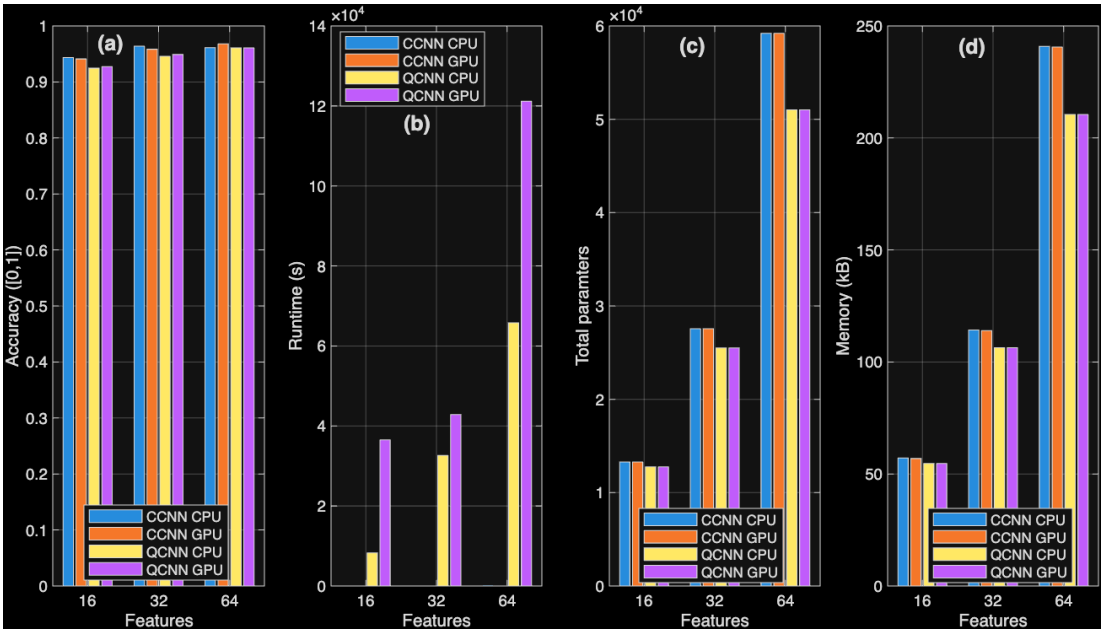


Fig. 5: Bar graph plot of (a) accuracy, (b) runtime (s), (c) total parameters, and (d) memory (kB) variation across the CCNN and QCNN with CPU and GPU at epoch 5 when varying features but keeping sample size fixed (i.e., 60000).

the sample size fixed (i.e., 60,000). In general, we observe that classification/identification accuracy increases with the increase of epochs, both for QCNN and CCNN.

Accuracy Comparison: In Figure 5a, we observe, in general, classification/identification accuracy improves with the increase of feature count. Additionally, while the classical CNN (CCNN) and quantum CNN (QCNN) achieve close accuracy, as the feature count increases, the accuracy gap between the CCNN and QCNN decreases from 0.0161 to 0.0071, consistent with our previous findings (Section IV-A1).

Runtime Comparison: In Figure 5b, we observe that the runtime of quantum CNN (QCNN) increases very fast with the increase of feature count, unlike the classical CNN (CCNN), which remains low and steady with the increase of feature count, varying 93.31s – 172.85s (CCNN CPU) and 94.87s – 98.22s (CCNN GPU). While the QCNN CPU takes 8,255.06s ($\approx$ 2h) to $\approx$ 65,785.89s ($\approx$ 18h), the QCNN GPU takes 36,535.4s ($\approx$ 10h) to $\approx$ 121168.92s ($\approx$ 33h), with the increase of feature count from 16 to 64. Thereby, QCNN CPU witnesses a higher jump ($\approx$ 88%) in runtime compared to a $\approx$ 70% jump in QCNN GPU, with the increase of feature count from 16 to 64, consistent with the exponential versus linear jump in runtime of CPU vs. GPU versions found in Section IV-A1.

Parameter Count Comparison: In Figure 5c, we observe that, in general, the total number of parameters increases with the increase of feature count across all four combinations. While increasing the feature count from 16 to 64, the accuracy gap between CCNN and QCNN drops from 0.0161 to 0.0071 (i.e., $\approx$ 56% drop), the difference in total number of parameters between CCNN and QCNN increases from 512 to 8192 (i.e., CCNN requires $\approx$ 94% more parameters than QCNN), making the QCNN a good choice at higher feature count, i.e., when parameter count is a major consideration.

TABLE II: Accuracy variation of QCNN and CCNN across different sample sizes and epochs for feature count 16

Models	Sample Size	Accuracy across 5 epochs				
		1	2	3	4	5
CCNN CPU	15000	0.8842	0.9065	0.9104	0.9163	0.9229
	30000	0.8999	0.9118	0.9116	0.9265	0.9311
	60000	0.9026	0.918	0.9355	0.9391	0.9434
CCNN GPU	15000	0.8596	0.8909	0.9024	0.9131	0.9203
	30000	0.8927	0.9117	0.9227	0.9272	0.9322
	60000	0.9157	0.9354	0.9385	0.9462	0.9442
QCNN CPU	15000	0.8539	0.8808	0.8911	0.899	0.9017
	30000	0.8868	0.9046	0.9056	0.9058	0.907
	60000	0.8921	0.9186	0.9105	0.9209	0.9245
QCNN GPU	15000	0.8539	0.8808	0.8911	0.899	0.9017
	30000	0.8646	0.8854	0.8966	0.9033	0.9109
	60000	0.8921	0.9186	0.9105	0.9209	0.9245

Memory Comparison: In Figure 5d, we observe that, in general, the memory requirement increases with the increase of feature count across all four combinations. While increasing the feature count from 16 to 64, the accuracy gap between CCNN and QCNN drops from 0.0161 to 0.0071 (i.e., $\approx$ 56% drop), the difference in memory requirement between CCNN and QCNN increases from 12705.1kB to 50777.4kB (i.e., CCNN requires $\approx$ 75% more memory than QCNN), making the QCNN a good choice at higher feature count, i.e., when memory is a major consideration. This finding is consistent with our finding in Figure 5c. Therefore, QCNN is a better choice than CCNN at higher feature counts and when memory and parameter counts are major considerations.

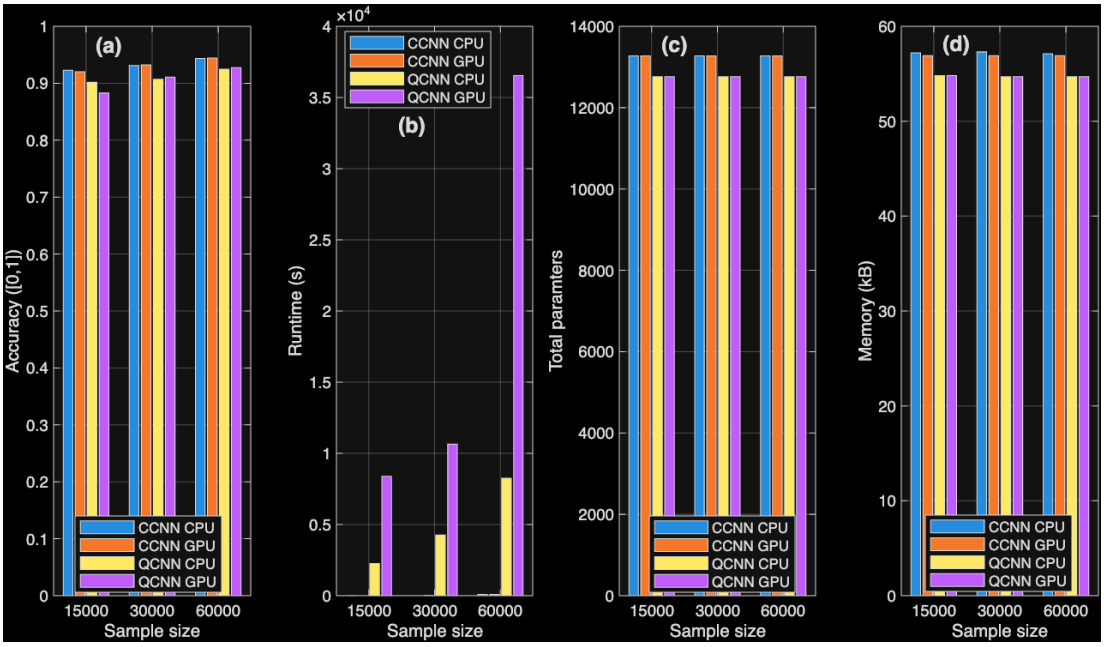


Fig. 6: Bar graph plot of (a) accuracy, (b) runtime (s), (c) total parameters, and (d) memory (kB) variation across the CCNN and QCNN with CPU and GPU at epoch 5 when varying sample size but keeping features fixed (i.e., 16).

2) Effect of Sample Size: Table II summarizes the performance of different models across different sample sizes (i.e., 15,000, 30,000, and 60,000), varying the epochs while keeping the feature count fixed (i.e., 16). In general, we observe that classification/identification accuracy increases with increasing epochs, both for QCNN and CCNN.

Accuracy Comparison: In Figure 6a, we observe, in general, classification/identification accuracy improves with the increase of sample size at a fixed feature count. Additionally, while the CCNN and QCNN achieve close accuracy, as the sample size increases from 15,000 to 60,000, the accuracy gap between them decreases from 0.0212 to 0.0169, consistent with our previous findings (Section IV-A2).

Runtime Comparison: In Figure 6b, we observe that the runtime of quantum CNN (QCNN) increases very fast with the increase of sample size, unlike the classical CNN (CCNN), which remains low and steady with the increase of feature count, varying 41.19s – 93.31s (CCNN CPU) and 33.08s – 94.27s (CCNN GPU). While the QCNN CPU takes 2,261.61s ($\approx$ 38 minutes) to $\approx$ 8,255.06s ($\approx$ 2h), the QCNN GPU takes 8,389.86s ($\approx$ 2h) to $\approx$ 36,535.4s ($\approx$ 10h), with the increase of feature count from 16 to 64. Thereby, QCNN CPU witnesses a similar jump ($\approx$ 73%) in runtime as QCNN GPU ($\approx$ 77%), with the increase in sample size from 15,000 to 60,000.

Parameter Count Comparison: In Figure 6c, we observe that parameter count remains unchanged with the increase of sample size across all four combinations, while QCNN requires 512 fewer parameters than the CCNN to achieve similar performance, i.e., accuracy. However, in Figure 6a, we found that the accuracy gap between QCNN and CCNN reduces with the increase of sample size. Therefore, the QCNN can be a good choice for larger sample sizes.

Memory Comparison: Similar to our findings in Figure 6c, we observe in Figure 6d that the memory requirement remains unchanged with increasing sample size across all four combinations, while QCNN requires 12705.1kB less memory than CCNN to achieve similar accuracy. However, in Figure 6a, we found that the accuracy gap between QCNN and CCNN reduces with the increase of sample size. Therefore, the QCNN can be a memory-efficient choice at larger sample sizes. This finding is consistent with our finding in Figure 6c. Therefore, QCNN is a better choice than CCNN at higher sample sizes and when memory and parameter counts are major considerations.

V. Conclusion

This study benchmarks classical and quantum machine learning models, i.e., SVMs and CNNs, across accuracy, runtime, parameter efficiency, and memory on the MNIST dataset. For the SVM-based models, QSVM consistently outperforms CSVM in accuracy, particularly at larger sample sizes (reaching $\sim$ 0.9 vs. $\sim$ 0.85 at 1,000 samples), though this comes at a higher computational cost. As QSVM runtime scales exponentially on CPU and linearly on GPU, GPU execution is the only practical choice for QSVMs. A feature count of 10 qubits and a sample size of 200 – 500 emerge as reasonable operating points that balance accuracy and runtime. For the neural network models, CCNN and QCNN achieve comparable classification accuracy (both exceeding 0.96 at 64 features and 60,000 samples), but QCNN offers substantially better parameter and memory efficiency, requiring $\sim$ 94% fewer parameters and $\sim$ 75% less memory than CCNN at higher feature counts, while suffering from longer runtimes.

Across both model families, quantum ML models outperform the classical ML models more as feature count or sample size increases, suggesting that quantum models offer the greatest relative advantage in high-data or high-dimensionality regimes.

Several promising avenues remain unexplored. First, experiments here were limited to simulated quantum hardware via PennyLane; evaluating these models on actual quantum hardware would expose the impact of noise, decoherence, and gate errors on both accuracy and runtime, factors absent from simulation. Second, scaling beyond the small qubit counts used here (up to 12 for QSVM, 4 per circuit for QCNN) is critical, as quantum advantage may emerge at higher dimensionalities that current simulators make prohibitively expensive to simulate. Third, the dataset was restricted to MNIST, a relatively simple benchmark; testing on more complex image datasets (e.g., CIFAR-10 or ImageNet subsets) would better stress-test the generalization capacity of quantum models. Fourth, exploring more expressive quantum circuit architectures, such as deeper variational circuits, alternative encoding strategies beyond angle embedding, or error-mitigation techniques, could improve accuracy while potentially reducing the runtime gap with classical counterparts. Finally, a systematic study of hybrid architectures that selectively apply quantum layers only where they provide measurable benefit could offer a pragmatic path toward quantum-classical co-design in real-world computer vision pipelines.

Looking ahead, this work directly supports the mission of the Alabama Transportation Institute (ATI) at The University of Alabama, to facilitate world-class, interdisciplinary transportation research that serves the state of Alabama and beyond, as well as the broader mandate of the National Center for Transportation Cybersecurity and Resiliency (TraCR), headquartered at Clemson University, which explicitly identifies artificial intelligence and quantum computing as transformative technologies for strengthening the cybersecurity and resiliency of the nation's transportation systems. ATI's strategic commitment to *ACES*^{2} *Mobility*, mobility that is **A**utomated, **C**onnected, **E**lectric, **S**hared, and **S**afe, demands robust, real-time computer vision capabilities for autonomous perception, object detection, traffic monitoring, and anomaly identification in connected and autonomous vehicles (CAVs) and intelligent transportation infrastructure. The benchmarking insights derived in this work, particularly the identification of practical operating regimes in terms of qubit count, sample size, and hardware environment where quantum models deliver measurable gains in accuracy and resource efficiency over classical counterparts, directly inform the design of next-generation, computationally efficient perception pipelines for such transportation applications. Specifically, the finding that QCNN requires $\sim$94% fewer parameters and $\sim$75% less memory than CCNN at higher feature counts, while achieving comparable accuracy, is especially relevant for resource-constrained onboard computing in CAVs and edge-deployed transportation sensors. Furthermore, the superior parameter efficiency of quantum models aligns with TraCR's goal of developing adaptive and resilient cyber-physical transportation systems, where lightweight yet accurate vision models can improve robustness against adversarial perturbations and reduce the attack surface of onboard AI inference systems. Our future plan is therefore to extend and apply the findings from this work directly to real-world transportation vision tasks supported by ATI and TraCR, including real-time traffic and road monitoring, pedestrian detection, vehicle classification at roadway intersections, and anomaly detection in connected infrastructure, working toward a safer, more efficient, and quantum-ready AI-assisted smart transportation ecosystem for Alabama and the nation.

ACKNOWLEDGEMENT

The author(s) gratefully acknowledge support from the Alabama Transportation Institute at The University of Alabama. This material is based upon the work supported by the National Center for Transportation Cybersecurity and Resiliency (TraCR) headquartered at Clemson University, Clemson, South Carolina, USA. Any opinions, findings, conclusions, and recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of TraCR, and the U.S. Government assumes no liability for the contents or use thereof.

REFERENCES

[1] R. Kaur and S. Singh, "A comprehensive review of object detection with deep learning," *Digital Signal Processing*, vol. 132, p. 103812, 2023.

[2] W. Cheung *et al.*, "Implicit IoT authentication using on-phone ANN models and breathing data," *Internet of Things*, vol. 24, p. 101003, 2023.

[3] S. V. Dibbo, A. Breuer, J. Moore, and M. Teti, "Improving robustness to model inversion attacks via sparse coding architectures," in *European Conference on Computer Vision*, pp. 117–136, Springer, 2024.

[4] Y. Kim *et al.*, "Environment knowledge-driven generic models to detect coughs from audio recordings," *IEEE Open Journal of Engineering in Medicine and Biology*, vol. 4, pp. 55–66, 2023.

[5] D. Amebley *et al.*, "Are neuro-inspired multi-modal vision-language models resilient to membership inference privacy leakage?," *arXiv preprint arXiv:2511.20710*, 2025.

[6] W. Cheung *et al.*, "Bag of on-phone ANNs to secure IoT objects using wearable and smartphone biometrics," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 3, pp. 1127–1138, 2023.

[7] "Grand View Research: Computer Vision Market Size, Share & Trends Analysis Report." Available:https://www.grandviewresearch.com/industry-analysis/computer-vision-market, Accessed: May 2026. [Online].

[8] R. Kharsa, A. Bouridane, and A. Amira, "Advances in quantum machine learning and deep learning for image classification: A survey," *Neurocomputing*, vol. 560, p. 126843, 2023.

[9] D. Peral-García, J. Cruz-Benito, and F. J. García-Peñalvo, "Systematic literature review: Quantum machine learning and its applications," *Computer Science Review*, vol. 51, p. 100619, 2024.

[10] "Grand View Research: Quantum Computing Market Size, Share & Trends Analysis Report." Available:https://www.grandviewresearch.com/industry-analysis/quantum-computing-market, Accessed: May 2026. [Online].

[11] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, p. 209–212, Mar. 2019.

[12] Y. Liu, W.-J. Li, X. Zhang, M. Lewenstein, G. Su, and S.-J. Ran, "Entanglement-based feature extraction by tensor network machine learning," *Frontiers in Applied Mathematics and Statistics*, vol. Volume 7 - 2021, 2021.

[13] "Alabama Transportation Institute (ATI)." Available:https://ati.ua.edu/about/, Accessed: May 2026. [Online].

[14] "National Center for Transportation Cybersecurity and Resiliency (TraCR)." Available:https://www.clemson.edu/cecas/tracr/, Accessed: May 2026. [Online].
[15] S. V. Dibbo, H. M. H. Babu, and L. Jamal, "An efficient design technique of a quantum divider circuit," in *2016 IEEE international symposium on circuits and systems (ISCAS)*, pp. 2102–2105, IEEE, 2016.
[16] S. Nokhwal, S. Nokhwal, S. Pahune, and A. Chaudhary, "Quantum generative adversarial networks: Bridging classical and quantum realms," 2023.
[17] A. Senokosov, A. Sedykh, A. Sagingalieva, B. Kyriacou, and A. Melnikov, "Quantum machine learning for image classification," *Machine Learning: Science and Technology*, vol. 5, no. 1, p. 015040, 2024.
[18] I. Cong, S. Choi, and M. D. Lukin, "Quantum convolutional neural networks," *Nature Physics*, vol. 15, no. 12, pp. 1273–1278, 2019.
[19] M. Parigi, M. Khosrojerdi, F. Caruso, and L. Banchi, "Analysis of quantum image representations for supervised classification," *AVS Quantum Science*, vol. 8, Jan. 2026.
[20] Y. Dang, N. Jiang, H. Hu, Z. Ji, and W. Zhang, "Image classification based on quantum KNN algorithm," *CoRR*, vol. abs/1805.06260, 2018.
[21] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
[22] A. Paszke, S. Gross, F. Massa, *et al.*, "Pytorch: An imperative style, high-performance deep learning library," 2019.
[23] V. Bergholm, J. Izaac, M. Schuld, *et al.*, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," 2022.
[24] T. pandas development team, "pandas-dev/pandas: Pandas," Feb. 2020.
[25] C. R. Harris, K. J. Millman, S. J. van der Walt, *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, Sept. 2020.
[26] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
[27] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, "Scikit-learn: Machine learning in Python," vol. 12, pp. 2825–2830, 2011.